

Tendências de Motores de Jogos

Semana 6

Data Tables, Data Assets, Structs e Enums

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 2 — Construir Sistemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender a separação entre dados de design e lógica de gameplay como problema universal de arquitetura
- Diferenciar Data Table (coleção tabular) de Data Asset (instância independente)
- Criar `DT_Items` tipada por `S_ItemData` e `E_ItemType`, conforme `PROJECT_ARCHITECTURE.md`
- Aplicar `DT_Items` a um Actor de coleta que reutiliza `BPI_Interactable` e o Event Dispatcher da Semana 5

Como a semana está organizada



Encontro 1

Data Table e Data Asset — criação de `DT_Items`



Encontro 2

Struct e Enum — tipagem de `DT_Items` + Actor de coleta + Checkpoint do Módulo 2

Tendências de Motores de Jogos

ENCONTRO 1

Data Table e Data Asset

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Se o mesmo item aparecer em três baús diferentes do nível, quantos lugares você precisaria editar para mudar seu nome?

Pense em onde essa informação deveria morar.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Data Table: coleção tabular de dados

- Estrutura tabular, editável como planilha, onde cada linha é uma instância de dado
- Ferramenta correta quando existem várias instâncias com os mesmos atributos
- `DT_Items` centraliza nome, descrição e ícone de cada item do Vertical Slice
- Existe de forma independente de qualquer Actor específico

DICA

Toda engine madura precisa de uma camada de dados que exista fora da lógica que a consome.

Data Asset: instância independente

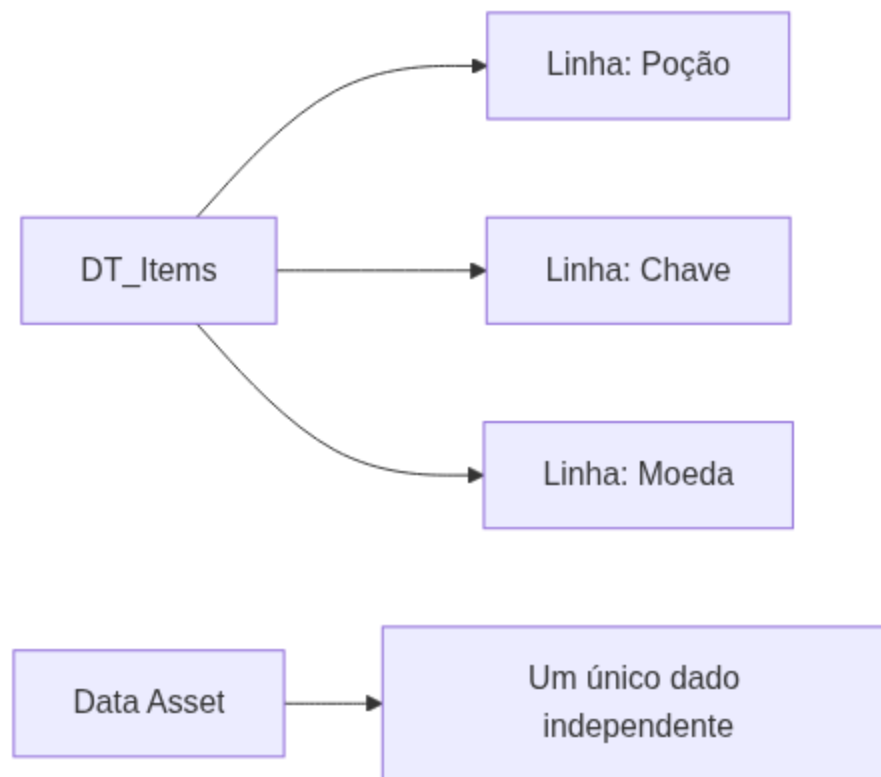
- Asset independente que representa um único dado, mais complexo ou não tabular por natureza
- Usado quando a informação não se organiza bem em linhas e colunas
- Nesta aula o foco é a Data Table, por ser a ferramenta correta para uma coleção homogênea de itens



Data Table e Data Asset resolvem o mesmo problema — dado fora do Actor — por caminhos diferentes.

Tendências de Motores de Jogos

Data Table × Data Asset



Dado desacoplado da lógica: Unreal × Unity

Unreal

Data Table já nasce orientada a coleção — uma única planilha com uma linha por item

Unity

ScriptableObject é um asset independente por instância; uma coleção exige organizar manualmente uma lista ou array desses assets

Demonstração: criando DT_Items

O professor cria `DT_Items` na subpasta `Data/DataTables/`, ainda com o tipo de linha padrão do editor, populando duas ou três linhas de exemplo.

Resultado esperado: tabela existindo de forma independente de qualquer Actor, pronta para receber tipagem no Encontro 2.

Imagem sugerida

Objetivo: mostrar a diferença conceitual entre Data Table e Data Asset.

Enquadramento: diagrama lado a lado, dois blocos.

Elementos presentes: à esquerda, uma planilha estilizada com várias linhas rotuladas "Item 1", "Item 2", "Item 3"; à direita, um único ícone de asset isolado.

Destaque visual: a Data Table concentra várias instâncias homogêneas; o Data Asset representa uma instância única.

Legenda sugerida: "Data Table: coleção tabular de itens homogêneos — Data Asset: instância única e independente."

Arquitetura: onde o dado mora

- `Data/DataTables/DT_Items` — nova subpasta temática do projeto
- `BPI_Interactable` e Event Dispatcher da Semana 5 permanecem intactos
- Base para a tipagem via Struct/Enum e para o Actor de coleta do Encontro 2

NA INDÚSTRIA

Nenhum sistema futuro — como o Inventário da Semana 10 — poderia consultar dados que não existem em um lugar central.

Boas práticas

✓ BOAS PRÁTICAS

- Tratar `DT_Items` como única fonte de verdade sobre os itens do Vertical Slice
- Nunca duplicar nome, descrição ou ícone em variáveis dentro de um Actor
- Renomear sempre o asset para `DT_Items`, nunca manter o nome padrão do editor

Laboratório do Encontro 1

Criar `DT_Items` na subpasta `Data/DataTables/`, com nomenclatura conforme `PROJECT_ARCHITECTURE.md` e ao menos uma linha de exemplo preenchida.

OBJETIVOS

Critério de sucesso: `DT_Items` existente, organizada e com ao menos uma linha preenchida, pronta para tipagem no Encontro 2.

Fechando o Encontro 1

- Data Table centraliza uma coleção de dados homogêneos; Data Asset representa uma instância independente
- `DT_Items` criada na subpasta `Data/DataTables/`, ainda sem tipagem forte
- Próximo encontro: dar forma e categoria a cada linha da tabela

Tendências de Motores de Jogos

ENCONTRO 2

Struct, Enum e o Actor de coleta

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos



Se outro colega do grupo digitar a categoria de um item de forma ligeiramente diferente na próxima linha, o sistema perceberia isso como erro?

Pense no que acontece quando um campo de categoria é apenas texto livre.

Struct: a forma do dado

- Agrupa campos relacionados sob um único tipo nomeado
- `S_ItemData` reúne nome, descrição, ícone e valor em uma estrutura
- Torna `DT_Items` fortemente tipada: toda linha passa a ter exatamente os mesmos campos, na mesma forma



DICA

Se a Data Table resolve "onde o dado mora", o Struct resolve "que forma esse dado deve ter".

Enum: categorias fechadas

- Restringe um campo a um conjunto fechado e nomeado de opções
- `E_ItemType` define categorias como Consumível, Chave ou Recurso
- Elimina ambiguidade e permite que sistemas futuros — como o Inventário — decidam com base na categoria, não em texto livre

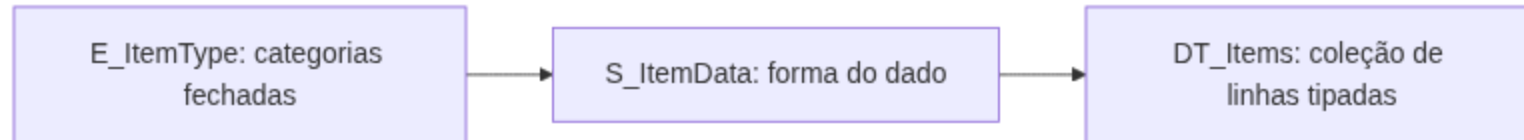


DICA

Nenhum campo de opções fechadas deveria ser texto livre.

Tendências de Motores de Jogos

Struct, Enum e Data Table juntos



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tipagem de dado: Unreal × Unity

Unreal

Struct definido em Blueprint gera colunas automaticamente na Data Table, editável nativamente no Data Table Editor

Unity

Classe serializável
(`[System.Serializable] class`) e `enum`
de C# resolvem o mesmo problema, com exibição no Inspector dependendo de como o campo é declarado

Demonstração: S_ItemData e E_ItemType

O professor cria `S_ItemData` (nome, descrição, ícone, valor, tipo) e `E_ItemType` (Consumível, Chave, Recurso), aplica o Struct como Row Structure de `DT_Items` e repopula a tabela.

Resultado esperado: `DT_Items` com um menu suspenso de categorias em vez de texto livre.

Demonstração: conectando ao Actor de coleta

O professor conecta `DT_Items` tipada a um Actor que já implementa `BPI_Interactable` : ao interagir, o Actor consulta a tabela via **Get Data Table Row** e dispara o Event Dispatcher da Semana 5 com os dados obtidos.

Por que: provar que o Actor de coleta não guarda dado — apenas um identificador de linha.

Imagem sugerida

Objetivo: mostrar o fluxo completo de consulta de dado no momento da interação.

Enquadramento: diagrama de fluxo horizontal, três blocos conectados por setas.

Elementos presentes: bloco 1 — "Jogador interage" (ícone de Actor de coleta); bloco 2 — "Get Data Table Row com ItemRowName" (ícone de tabela); bloco 3 — "Event Dispatcher dispara com dados de S_ItemData" (ícone de sino).

*Destaque visual: a seta entre bloco 1 e bloco 2 representa **BPI_Interactable**; a seta entre bloco 2 e bloco 3 representa a consulta tipada retornando **S_ItemData**.*

Legenda sugerida: "Do clique do jogador ao dado tipado: interação, consulta e notificação em três passos."

Arquitetura: roadmap atualizado

- `Data/Structs_Enums/S_ItemData` e `E_ItemType` — novas classes centralizadas
- `Blueprints/Interactables/BP_Chest` ou `BP_Pickup` reutiliza `BPI_Interactable` e o Event Dispatcher da Semana 5
- Linha "DT_Items (Data Table + Struct + Enum)" do roadmap encerrada; "BP_Chest, BP_Pickup" iniciada

NA INDÚSTRIA

O Actor de coleta guarda apenas um identificador de linha — nunca os dados do item em si.

Boas práticas

✓ BOAS PRÁTICAS

- Manter `S_ItemData` apenas com campos que qualquer item tem em comum — exceções específicas pertencem a módulos futuros
- Structs e Enums usados por mais de um sistema ficam em `Data/Structs_Enums/`, nunca duplicados dentro de um Actor
- O Actor de coleta permanece enxuto: sua única responsabilidade é consultar e notificar, nunca reagir

Laboratório do Encontro 2

Criar `S_ItemData` e `E_ItemType`, aplicar como Row Structure de `DT_Items` e conectar a tabela a um Actor de coleta que reutiliza `BPI_Interactable` e o Event Dispatcher da Semana 5.

OBJETIVOS

Critério de sucesso: `DT_Items` tipada, consulta funcionando na interação, Event Dispatcher disparado com os dados obtidos.

Desafio: seu próprio conjunto de itens

Cada grupo modela seu conjunto de itens coletáveis (baús, moedas, recursos ou categoria própria), com liberdade sobre categorias e atributos, desde que o Actor de coleta reutilize `BPI_Interactable` e o Event Dispatcher da Semana 5.

ATENÇÃO

Sem gabarito único de categorias ou atributos — a única exigência é a reutilização do par Interface + Event Dispatcher.

Resumo da Semana 6

- Data Table centraliza dado; Struct dá forma; Enum fecha categorias — três peças complementares
- `DT_Items` tipada e populada, conectada a um Actor de coleta funcional no Vertical Slice
- Dado de design e lógica de gameplay (Interface + Event Dispatcher) integrados pela primeira vez
- Checkpoint de progresso do Módulo 2 concluído

Próximos passos

DICA

A Semana 7 integra, em um único fluxo jogável, todos os sistemas construídos desde a Semana 4 (GameMode, GameState, PlayerController, GameInstance, Interfaces, Event Dispatchers, Data Tables, Structs, Enums), encerrando a Unidade II com Code Review e Playtest coletivo. O SaveGame Object da Semana 7 precisará serializar o progresso construído até aqui.

Leitura recomendada: Gameplay Framework in Unreal Engine (Epic Games Documentation).